

LAPORAN PRAKTIKUM
POSTTEST (4)
ALGORITMA PEMROGRAMAN LANJUT



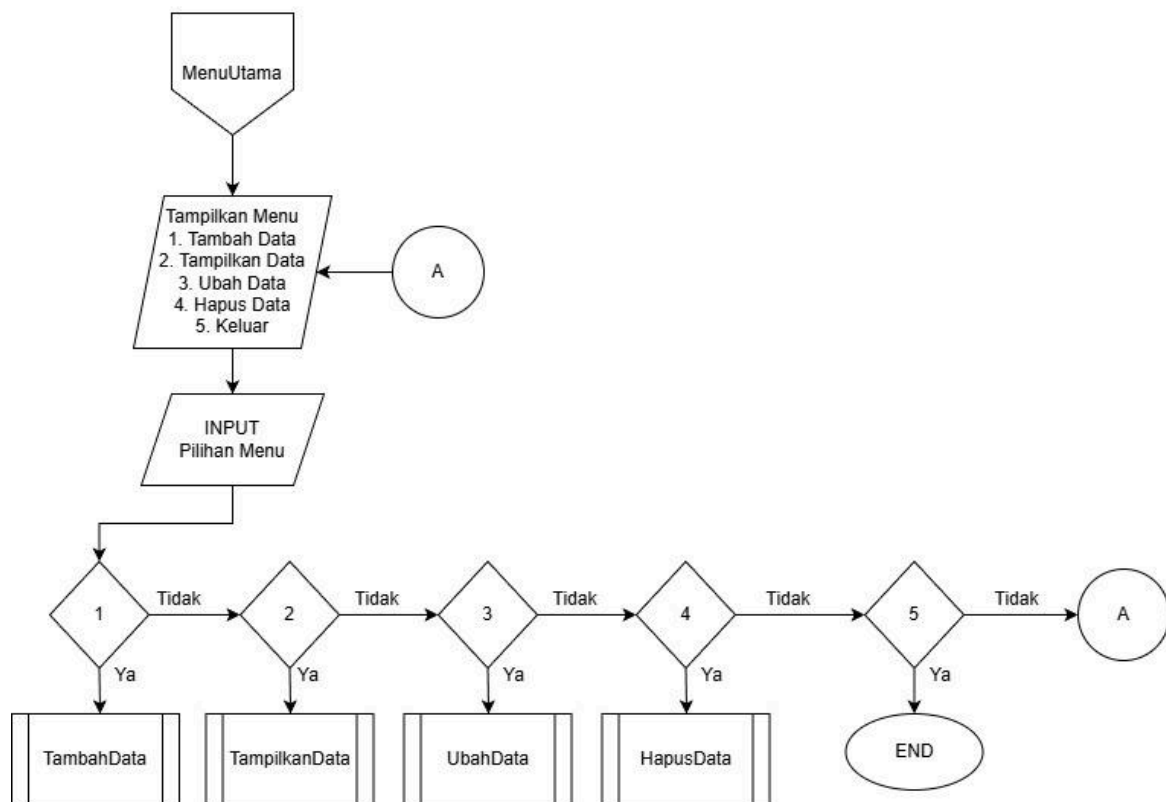
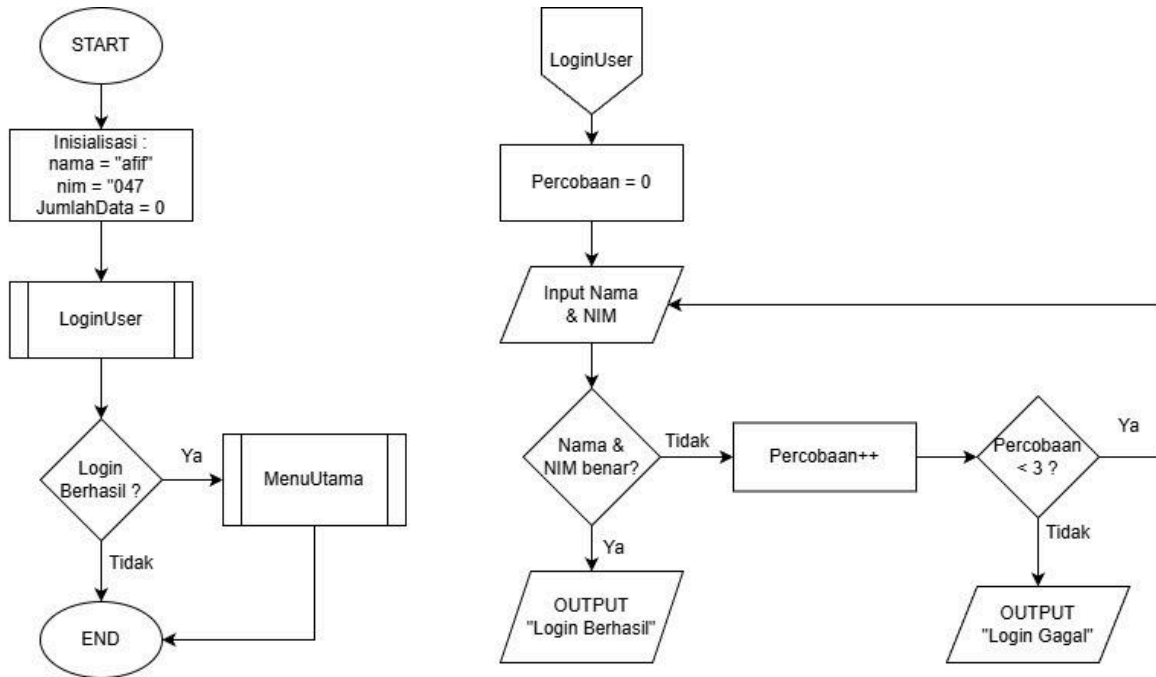
Disusun oleh:

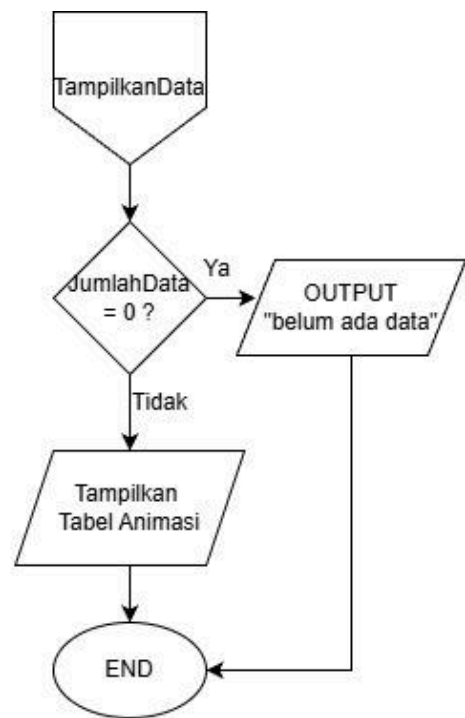
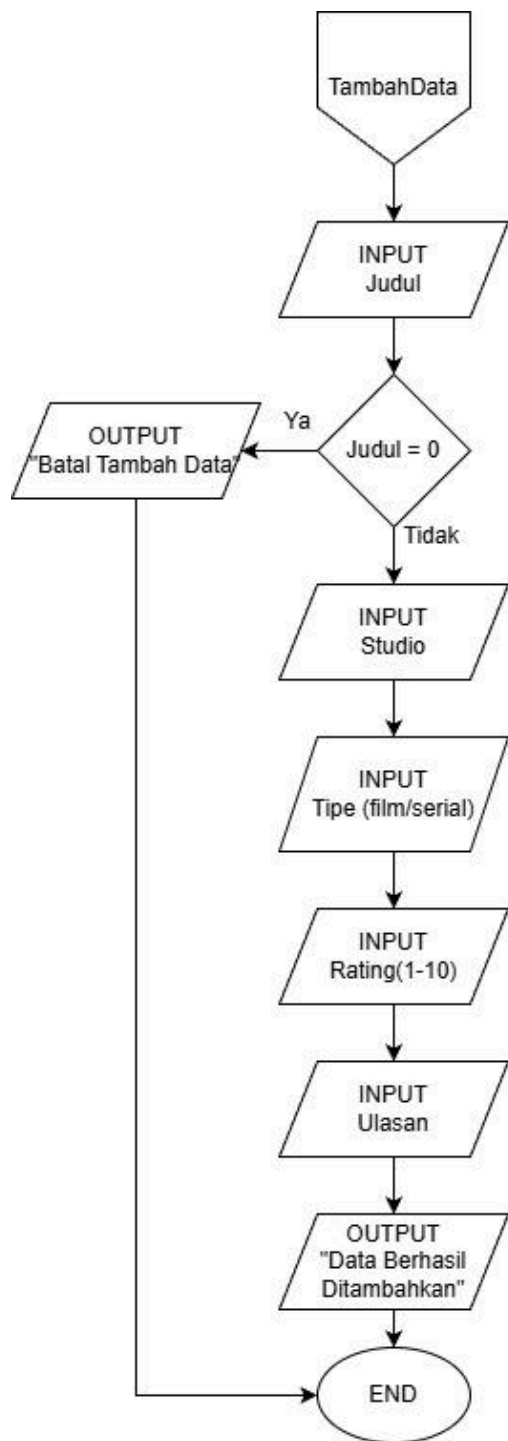
Nama (2509106047)

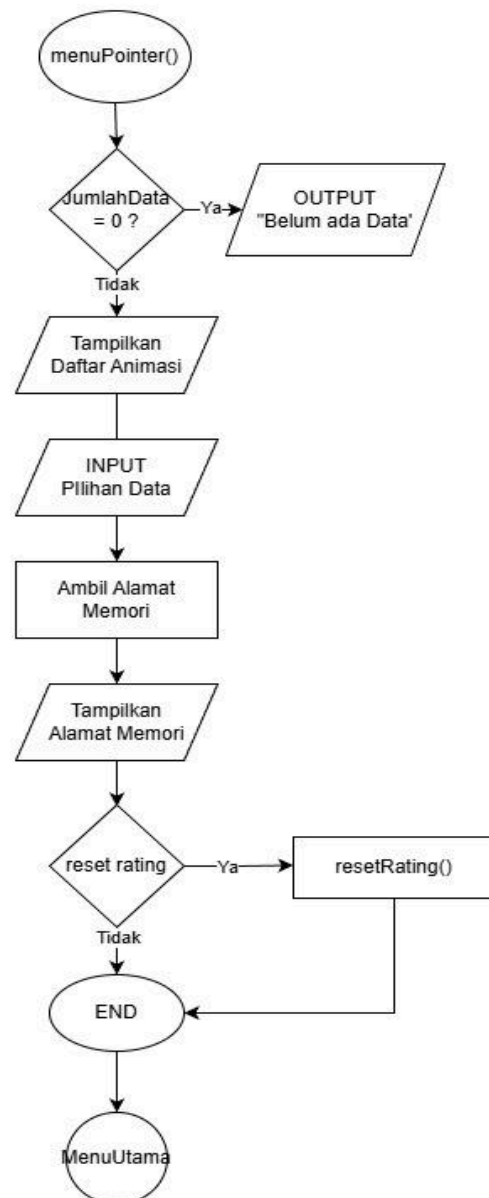
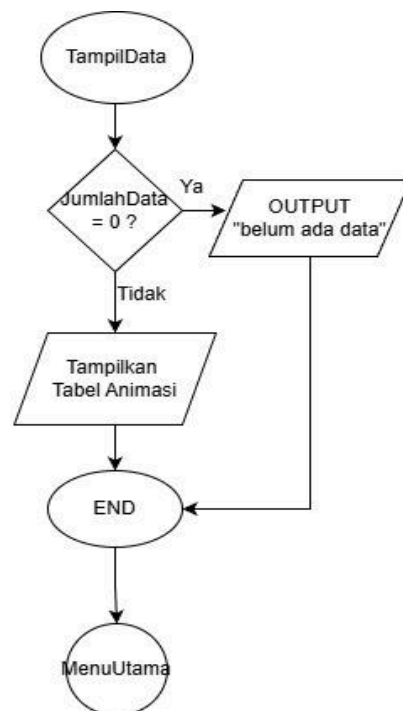
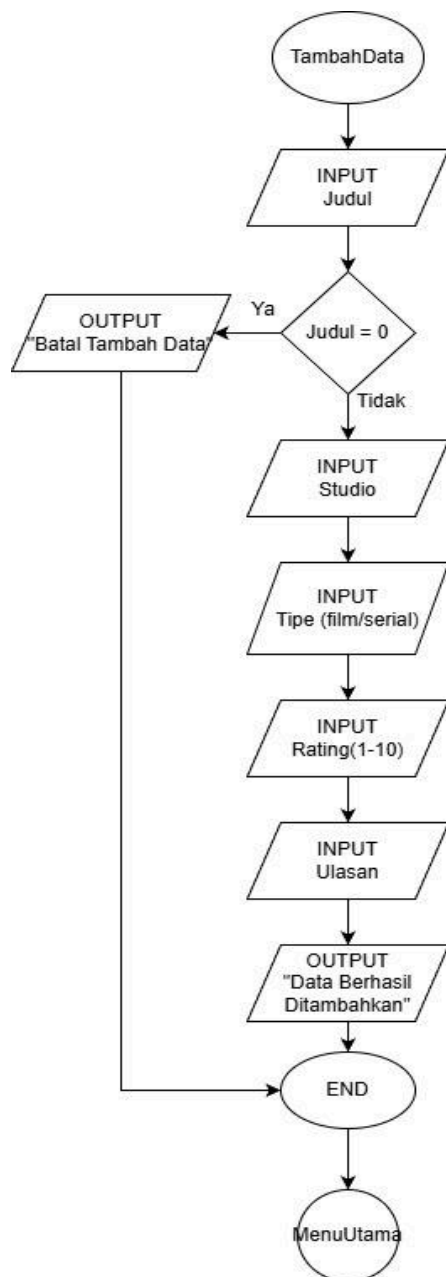
Kelas (B1 '25)

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2026

1. Flowchart







Penjelasan Flowchart

1. Start

Program dimulai

2. Login

Program meminta pengguna memasukkan nama dan NIM untuk login.

Lalu program mengecek apakah nama dan NIM yang dimasukkan sudah benar atau belum.

Jika benar, pengguna bisa masuk ke menu utama.

Jika salah, pengguna akan diminta login lagi.

3. Cek Percobaan Login

Program mengecek apakah percobaan login masih kurang dari 3 kali.

Jika masih kurang dari 3, pengguna bisa mencoba login lagi.

Jika sudah 3 kali gagal, program akan langsung berhenti.

4. Menu Utama

Jika login berhasil, program akan menampilkan menu utama yang berisi beberapa pilihan:

1. Tambah Data Animasi

2. Tampilkan Data Animasi

3. Ubah Data Animasi

4. Hapus Data Animasi

5. Keluar

Menu ini akan terus muncul berulang sampai pengguna memilih keluar.

Menu 1 - Tambah Data

Pengguna memasukkan data animasi seperti judul, studio, tipe animasi, rating, dan ulasan.

Data tersebut kemudian disimpan ke dalam array.

Menu 2 - Tampilkan Data

Program akan menampilkan semua data animasi dalam bentuk tabel.

Menu 3 - Ubah Data

Program menampilkan daftar data animasi terlebih dahulu.

Lalu pengguna memilih nomor data yang ingin diubah, kemudian memasukkan data yang baru.

Menu 4 - Hapus Data

Program menampilkan daftar data animasi.

Pengguna memilih nomor data yang ingin dihapus, lalu data tersebut akan dihapus dari daftar.

Menu 5 - Pointer

Program Menampilkan Daftar Animasi.

Pengguna dapat memilih animasi dan menampilkan alamat dari data tersebut, pengguna juga bisa reset rating.

Menu 6 - Keluar

Mengakhiri Program

6. End

Program selesai ketika pengguna memilih menu keluar atau gagal login 3 kali.

2. Deskripsi Singkat Program

Program ini dibuat untuk mengelola data animasi menggunakan bahasa pemrograman C++ dengan fitur tambah, lihat, ubah, dan hapus data. Program memiliki bagian penting seperti sistem login untuk memverifikasi pengguna serta penggunaan struktur data untuk menyimpan informasi animasi seperti judul, studio, tipe, rating, dan ulasan. Alur program dimulai dari proses login, kemudian pengguna masuk ke menu utama dan dapat memilih fitur CRUD untuk mengelola data animasi yang tersimpan dalam array. Program telah ditambahkan penggunaan pointer. Pengguna dapat melihat alamat dari data di dalam program. Dengan adanya program ini, pengguna dapat mencatat dan mengelola daftar animasi secara sederhana melalui tampilan menu di terminal.

3. Source Code

STRUCT

```
struct DataUser{
    string nama;
    string nim;
};

struct DataAnimasi{
    string judul;
    string studio;
    string tipe;
    int rating;
    string ulasan;
};
```

Struct digunakan untuk mengelompokkan beberapa variabel menjadi satu tipe data. DataUser menyimpan data login pengguna yaitu nama dan nim. DataAnimasi menyimpan data animasi yang terdiri dari judul, studio, tipe, rating, dan ulasan.

FUNGSI

```
int hitungJumlahData(int jumlahData){
    if(jumlahData==0)
        return 0;
    else
        return 1 + hitungJumlahData(jumlahData-1);
}

bool verifikasiLogin(DataUser user, string inputNama, string inputNim){
    if(inputNama==user.nama && inputNim==user.nim)
        return true;
    else
        return false;
}
```

Fungsi adalah blok kode yang mengembalikan nilai. hitungJumlahData menggunakan rekursif untuk menghitung total data animasi yang tersimpan. verifikasiLogin mengembalikan nilai true jika nama dan nim yang diinput sesuai dengan data user, dan false jika tidak.

PROSEDUR

```
void login(DataUser user, bool &statusLogin){

    int jumlahPercobaan=0;
    string inputNama, inputNim;

    while(jumlahPercobaan<3){
        system("cls");

        cout<<"==== LOGIN =====<<endl;
        cout<<"Nama : ";
        cin>>inputNama;

        cout<<"NIM : ";
        cin>>inputNim;

        if(verifikasiLogin(user, inputNama, inputNim)){
            statusLogin=true;
            cout<<"Login berhasil!"<<endl;
            system("pause");
            break;
        }
        else{
            jumlahPercobaan++;
            cout<<"Login gagal!"<<endl;
            cout<<"Sisa percobaan : "<<3-jumlahPercobaan<<endl;
            system("pause");
        }
    }
}

void tampilMenu(){
    system("cls");

    cout<<"==== MENU UTAMA =====<<endl;
    cout<<"1. Tambah Data Animasi"<<endl;
    cout<<"2. Lihat Data Animasi"<<endl;
    cout<<"3. Ubah Data Animasi"<<endl;
    cout<<"4. Hapus Data Animasi"<<endl;
    cout<<"5. Lihat Alamat Memori Data"<<endl;
    cout<<"6. Keluar"<<endl;

    cout<<"Pilih menu : ";
}
```

Prosedur adalah blok kode bertipe void yang tidak mengembalikan nilai. Prosedur login menangani proses login pengguna dengan maksimal 3 kali percobaan. Parameter statusLogin menggunakan address-of operator & sehingga perubahan nilainya langsung mempengaruhi variabel asli di main. Prosedur tampilMenu menampilkan pilihan menu utama program.

POINTER

```
void resetRating(int &rating){
    rating = 0;
    cout<<"Rating telah direset!"<<endl;
    cout<<"Alamat memori rating : "<<&rating<<endl;
}

void updateRating(int *rating, int ratingBaru){
    cout<<"Alamat memori rating di fungsi : "<<rating<<endl;
    *rating = ratingBaru;
    cout<<"Rating berhasil diupdate!"<<endl;
}

void tampilAlamatMemori(DataAnimasi *animasiPtr){
    cout<<"=== Alamat Memori Data Animasi ==="<<endl;
    cout<<"Alamat struct   : "<<animasiPtr<<endl;
    cout<<"Alamat judul     : "<<&animasiPtr->judul<<endl;
    cout<<"Alamat studio      : "<<&animasiPtr->studio<<endl;
    cout<<"Alamat tipe        : "<<&animasiPtr->tipe<<endl;
    cout<<"Alamat rating      : "<<&animasiPtr->rating<<endl;
    cout<<"Alamat ulasan      : "<<&animasiPtr->ulasan<<endl;
}

void menuPointer(DataAnimasi dataAnimasi[], int jumlahData){

    system("cls");

    if(jumlahData==0){
        cout<<"Belum ada data"<<endl;
        system("pause");
        return;
    }

    cout<<"=== Fitur Pointer ==="<<endl<<endl;

    for(int i=0; i<jumlahData; i++){
        cout<<i+1<<" ". <<dataAnimasi[i].judul<<endl;
    }

    int pilihanData;

    cout<<"Pilih nomor data : ";
    cin>>pilihanData;

    if(pilihanData>=1 && pilihanData<=jumlahData){

        int indeks = pilihanData-1;

        DataAnimasi *animasiPtr = &dataAnimasi[indeks];
        tampilAlamatMemori(animasiPtr);
    }
}
```

```

        cout<<endl;
        cout<<"Apakah ingin reset rating ke 0? (y/n) : ";
        char pilih;
        cin>>pilih;

        if(pilih=='y' || pilih=='Y'){
            cout<<"Alamat memori rating di main :
"<<&dataAnimasi[indeks].rating<<endl;
            resetRating(dataAnimasi[indeks].rating);
        }
    }
    else{
        cout<<"Data tidak ditemukan"<<endl;
    }

    system("pause");
}

```

Terdapat tiga penggunaan pointer. Pertama, resetRating menggunakan address-of operator & sebagai parameter sehingga nilai rating langsung diubah di alamat memori aslinya. Kedua, updateRating menggunakan dereference operator * sebagai parameter untuk menerima alamat memori rating lalu mengubah nilainya menggunakan *rating. Ketiga, tampilAlamatMemori menggunakan pointer pada struct dengan operator untuk mengakses dan menampilkan alamat memori tiap atribut dari struct DataAnimasi. Fungsi menuPointer menggabungkan ketiganya dalam satu menu.

CREATE

```

void tambahData(DataAnimasi dataAnimasi[], int &jumlahData){

    system("cls");
    cout<<"== Tambah Data Animasi =="<<endl;
    cout<<"(Kosongkan judul untuk batal)"<<endl<<endl;

    cout<<"Judul : ";
    getline(cin, dataAnimasi[jumlahData].judul);

    if(dataAnimasi[jumlahData].judul==""){
        cout<<"Tambah data dibatalkan"<<endl;
        system("pause");
        return;
    }

    cout<<"Studio : ";
    getline(cin, dataAnimasi[jumlahData].studio);

    while(true){
        cout<<"Tipe (film / serial) : ";
        cin>>dataAnimasi[jumlahData].tipe;
    }
}

```

```

        if(dataAnimasi[jumlahData].tipe=="film" ||
dataAnimasi[jumlahData].tipe=="serial")
            break;

        cout<<"Input salah!"<<endl;
    }

    while(true){
        cout<<"Rating (1-10) : ";
        cin>>dataAnimasi[jumlahData].rating;

        if(dataAnimasi[jumlahData].rating>=1 &&
dataAnimasi[jumlahData].rating<=10)
            break;

        cout<<"Rating harus 1 sampai 10"<<endl;
    }

    cin.ignore();

    cout<<"Ulasan : ";
    getline(cin, dataAnimasi[jumlahData].ulasan);

    jumlahData++;

    cout<<"Data berhasil ditambahkan"<<endl;
    system("pause");
}

```

Fungsi tambahData digunakan untuk menambah data animasi baru ke dalam array. Jika judul dikosongkan maka proses tambah data akan dibatalkan. Terdapat validasi pada input tipe agar hanya menerima nilai film atau serial, dan validasi rating agar hanya menerima angka 1 sampai 10. Data disimpan ke index jumlahData lalu jumlahData ditambah satu setelah data berhasil dimasukkan.

READ

```

void tampilData(DataAnimasi dataAnimasi[], int jumlahData){

    system("cls");

    cout<<"=== Daftar Animasi ==="<<endl;
    cout<<"===== "<<endl;

    if(jumlahData==0){
        cout<<"Belum ada data animasi"<<endl;
    }
    else{
        for(int i=0; i<jumlahData; i++){

            cout<<"[ "<<i+1<<" ]"<<endl;

```

```

        cout<<left;
        cout<<setw(10)<<"Judul"<<": " <<dataAnimasi[i].judul<<endl;
        cout<<setw(10)<<"Studio"<<": " <<dataAnimasi[i].studio<<endl;
        cout<<setw(10)<<"Tipe"<<": " <<dataAnimasi[i].tipe<<endl;
        cout<<setw(10)<<"Rating"<<": " <<dataAnimasi[i].rating<<"/10"<<endl;
        cout<<setw(10)<<"Ulasan"<<": " <<dataAnimasi[i].ulasan<<endl;
        cout<<"-----"<<endl;
    }

    int totalData = hitungJumlahData(jumlahData);
    cout<<"Total animasi : " <<totalData<<endl;
}

system("pause");
}

```

Fungsi tampilData menampilkan seluruh data animasi dalam format card per data menggunakan setw dari library iomanip agar tampilannya rapi. Jika belum ada data maka akan muncul pesan bahwa data masih kosong. Total data dihitung menggunakan fungsi rekursif hitungJumlahData.

UPDATE

```

void ubahData(DataAnimasi dataAnimasi[], int jumlahData){

    system("cls");

    if(jumlahData==0){
        cout<<"Belum ada data"<<endl;
        system("pause");
        return;
    }

    cout<<"== Ubah Data Animasi =="<<endl;
    cout<<"(Ketik 0 untuk batal)"<<endl<<endl;

    for(int i=0; i<jumlahData; i++){
        cout<<i+1<<". " <<dataAnimasi[i].judul<<endl;
    }

    int pilihanData;

    cout<<"Pilih nomor data : ";
    cin>>pilihanData;
    cin.ignore();

    if(pilihanData==0){
        cout<<"Ubah data dibatalkan"<<endl;
        system("pause");
        return;
    }
}

```

```

if(pilihanData>=1 && pilihanData<=jumlahData){

    int indeks = pilihanData-1;

    cout<<"Judul baru (lama: "<<dataAnimasi[indeks].judul<<" ) : ";
    getline(cin, dataAnimasi[indeks].judul);

    cout<<"Studio baru (lama: "<<dataAnimasi[indeks].studio<<" ) : ";
    getline(cin, dataAnimasi[indeks].studio);

    cout<<"Tipe baru (lama: "<<dataAnimasi[indeks].tipe<<" ) : ";
    cin>>dataAnimasi[indeks].tipe;

    int ratingBaru;
    cout<<"Rating baru (lama: "<<dataAnimasi[indeks].rating<<" ) : ";
    cin>>ratingBaru;
    updateRating(&dataAnimasi[indeks].rating, ratingBaru);

    cin.ignore();

    cout<<"Ulasan baru (lama: "<<dataAnimasi[indeks].ulasan<<" ) : ";
    getline(cin, dataAnimasi[indeks].ulasan);

    cout<<"Data berhasil diubah"<<endl;
}
else{
    cout<<"Data tidak ditemukan"<<endl;
}

system("pause");
}

```

Fungsi ubahData digunakan untuk mengubah data animasi yang sudah ada. Pengguna memilih nomor data yang ingin diubah, jika memilih 0 maka proses dibatalkan. Nilai lama ditampilkan sebagai referensi sebelum pengguna memasukkan nilai baru. Pada bagian update rating digunakan fungsi updateRating dengan mengirimkan alamat memori rating menggunakan operator &.

DELETE

```
void hapusData(DataAnimasi dataAnimasi[], int &jumlahData){

    system("cls");

    if(jumlahData==0){
        cout<<"Belum ada data"<<endl;
        system("pause");
        return;
    }

    cout<<"=== Hapus Data Animasi ==="<<endl;
    cout<<"(Ketik 0 untuk batal)"<<endl<<endl;

    for(int i=0; i<jumlahData; i++){
        cout<<i+1<<" ". "<<dataAnimasi[i].judul<<endl;
    }

    int pilihanData;

    cout<<"Pilih nomor data : ";
    cin>>pilihanData;

    if(pilihanData==0){
        cout<<"Hapus data dibatalkan"<<endl;
        system("pause");
        return;
    }

    if(pilihanData>1 && pilihanData<=jumlahData){

        cout<<"Yakin hapus \""<<dataAnimasi[pilihanData-1].judul<<"\"? (y/n) : ";

        char konfirmasi;
        cin>>konfirmasi;

        if(konfirmasi=='n' || konfirmasi=='N'){
            cout<<"Hapus data dibatalkan"<<endl;
            system("pause");
            return;
        }

        for(int i=pilihanData-1; i<jumlahData-1; i++){
            dataAnimasi[i] = dataAnimasi[i+1];
        }

        jumlahData--;
        cout<<"Data berhasil dihapus"<<endl;
    }
    else{
```

```

        cout<<"Data tidak ditemukan"<<endl;
    }

    system("pause");
}

```

Fungsi hapusData digunakan untuk menghapus data animasi. Pengguna memilih nomor data yang ingin dihapus, jika memilih 0 maka proses dibatalkan. Sebelum dihapus pengguna diminta konfirmasi terlebih dahulu. Proses hapus dilakukan dengan menggeser semua data di belakang index yang dihapus ke depan satu posisi, lalu jumlahData dikurangi satu.

MAIN

```

int main(){

    DataUser user;
    user.nama="afif";
    user.nim="047";

    DataAnimasi dataAnimasi[50];
    int jumlahData=0;
    bool statusLogin=false;

    // data dummy
    dataAnimasi[0].judul    = "Avatar The Last Airbender";
    dataAnimasi[0].studio   = "Nickelodeon";
    dataAnimasi[0].tipe     = "serial";
    dataAnimasi[0].rating   = 10;
    dataAnimasi[0].ulasan   = "Serial animasi terbaik";

    dataAnimasi[1].judul    = "Gravity Falls";
    dataAnimasi[1].studio   = "Disney Television";
    dataAnimasi[1].tipe     = "serial";
    dataAnimasi[1].rating   = 10;
    dataAnimasi[1].ulasan   = "Misteri dan humor yang sangat seru";

    dataAnimasi[2].judul    = "The Incredibles";
    dataAnimasi[2].studio   = "Pixar";
    dataAnimasi[2].tipe     = "film";
    dataAnimasi[2].rating   = 9;
    dataAnimasi[2].ulasan   = "Film superhero keluarga yang sangat bagus";

    jumlahData=3;

    login(user, statusLogin);

    if(statusLogin==false){
        cout<<"Program berhenti karena gagal login"<<endl;
        return 0;
    }
}

```

```

int pilihanMenu;

do{
    tampilMenu();

    cin>>pilihanMenu;
    cin.ignore();

    switch(pilihanMenu){

        case 1:
            tambahData(dataAnimasi, jumlahData);
            break;

        case 2:
            tampilData(dataAnimasi, jumlahData);
            break;

        case 3:
            ubahData(dataAnimasi, jumlahData);
            break;

        case 4:
            hapusData(dataAnimasi, jumlahData);
            break;

        case 5:
            menuPointer(dataAnimasi, jumlahData);
            break;

        case 6:
            cout<<"Program selesai"<<endl;
            break;

        default:
            cout<<"Menu tidak valid"<<endl;
            system("pause");
    }

}while(pilihanMenu!=6);

return 0;
}

```

Fungsi main adalah titik awal program berjalan. Di sini dideklarasikan variabel user, array dataAnimasi berkapasitas 50, dan 3 data dummy animasi barat. Setelah login berhasil, program masuk ke perulangan do-while yang menampilkan menu dan memanggil fungsi sesuai pilihan pengguna hingga memilih keluar.

4. Hasil Output

```
===== LOGIN =====  
Nama : afif  
NIM : 047
```

Gambar 4.1 Login

```
===== MENU UTAMA =====  
1. Tambah Data Animasi  
2. Tampilkan Data Animasi  
3. Ubah Data Animasi  
4. Hapus Data Animasi  
5. Keluar  
Pilih menu : 
```

Gambar 4.2 Menu Utama

```
=== Tambah Data Animasi ===  
(Kosongkan judul untuk batal)  
  
Judul : Avatar The Last Airbender  
Studio : Nickelodeon  
Tipe (film / serial) : serial  
Rating (1-10) : 10  
Ulasan : serial animasi favorit  
Data berhasil ditambahkan  
Press any key to continue . . .
```

Gambar 4.3 Create

```

=== Daftar Animasi ===
=====
[ 1 ]
Judul      : Avatar The Last Airbender
Studio     : Nickelodeon
Tipe       : serial
Rating     : 10/10
Ulasan     : Serial animasi terbaik
-----
[ 2 ]
Judul      : Gravity Falls
Studio     : Disney Television
Tipe       : serial
Rating     : 10/10
Ulasan     : Misteri dan humor yang sangat seru
-----
[ 3 ]
Judul      : The Incredibles
Studio     : Pixar
Tipe       : film
Rating     : 9/10
Ulasan     : Film superhero keluarga yang sangat bagus
-----
Total animasi : 3
Press any key to continue . . . 

```

Gambar 4.4 Read

```

=== Ubah Data Animasi ===
(Ketik 0 untuk batal)

1. Avatar The Last Airbender
Pilih nomor data : 0
Ubah data dibatalkan
Press any key to continue . . . 

```

Gambar 4.5 Update

```

=== Hapus Data Animasi ===
(Ketik 0 untuk batal)

1. Avatar The Last Airbender
Pilih nomor data : 0
Hapus data dibatalkan
Press any key to continue . . .

```

Gambar 4.6 Delete

```

=== Fitur Pointer ===

1. Avatar The Last Airbender
2. Gravity Falls
3. The Incredibles
Pilih nomor data : 1
=== Alamat Memori Data Animasi ===
Alamat struct : 0x941dbfdf30
Alamat judul : 0x941dbfdf30
Alamat studio : 0x941dbfdf50
Alamat tipe : 0x941dbfdf70
Alamat rating : 0x941dbfdf90
Alamat ulasan : 0x941dbfdf98

```

Gambar 4.7 Pointer

```

===== MENU UTAMA =====
1. Tambah Data Animasi
2. Tampilkan Data Animasi
3. Ubah Data Animasi
4. Hapus Data Animasi
5. Keluar

Pilih menu : 5
Program selesai.
PS D:\Praktikum-APL\Post-test\Post-Test-APL-2>

```

Gambar 4.8 Keluar program

5. Langkah-langkah GIT

(Berikan screenshot dan jelaskan secara ringkas fungsi dari yang kalian ketik)

5.1 GIT Add

```
PS D:\Praktikum-APL> git add .
```

Digunakan untuk menambahkan file ke staging area. Artinya, file tersebut dipersiapkan untuk disimpan (di-commit) ke dalam repository.

5.2 GIT Commit

```
PS D:\Praktikum-APL> git commit -m "Finish Post Test 1"
[main (root-commit) 4d08deb] Finish Post Test 1
 2 files changed, 104 insertions(+)
 create mode 100644 Post-test/2509106047-AhmadAfifAdiyatma-PT-1.cpp
 create mode 100644 Post-test/2509106047-AhmadAfifAdiyatma-PT-1.exe
```

Digunakan untuk menyimpan perubahan yang sudah ada di staging area ke dalam repository dengan pesan commit. Commit ini seperti “titik penyimpanan” versi proyek.

5.3 GIT Push

```
PS D:\Praktikum-APL> git push -u origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 667.55 KiB | 4.04 MiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Ahmad-Afif-4707/Praktikum-APL.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS D:\Praktikum-APL> █
```

Digunakan untuk mengirim (mengunggah) commit dari repository lokal ke repository remote, misalnya ke GitHub, agar bisa diakses atau dibagikan secara online.